

Análise Quantitativa do Trade-off entre Especialização e Generalização em LLMs via Fine-Tuning

PPGINF528 – Tópicos Especiais em Recuperação da Informação – Trabalho Prático 2

Giovanni Escóssio Zanardo
Instituto de Computação
Universidade Federal do Amazonas (UFAM)
Manaus, Brasil
gio.zanardo@icomp.ufam.edu.br

Resumo—Avaliamos empiricamente o *trade-off* entre especialização e generalização decorrente do fine-tuning de Modelos de Linguagem de Grande Porte (LLMs). Aplicamos QLoRA sobre o Qwen2.5-3B-Instruct na tarefa de Text-to-SQL com o benchmark Spider, comparando o desempenho do modelo-base com duas configurações de hiperparâmetros distintas, e medindo a regressão de capacidade em uma suíte de 150 questões do MMLU em formato 5-shot, distribuídas em três categorias. A configuração de melhor desempenho atinge 64,02% de Execution Accuracy contra 49,13% do baseline, um ganho absoluto de 14,89 pontos percentuais e relativo de 30,3%. Contrariando a hipótese de regressão de capacidade, a acurácia agregada no MMLU sobe de 64% para 68%. Discutimos os fatores arquiteturais que explicam a ausência de esquecimento catastrófico nesse cenário, incluindo a baixa fração de parâmetros treináveis (0,24%) e o congelamento dos pesos do modelo-base via quantização em 4-bit. Apresentamos ainda uma análise de erros que evidencia como o *fine-tuning* deslocou o perfil de falhas do modelo de erros sintáticos grosseiros para erros semânticos mais sutis.

I. INTRODUÇÃO

O *fine-tuning* (FT) de Modelos de Linguagem de Grande Porte (LLMs) consolidou-se como a principal técnica de especialização de modelos pré-treinados para tarefas de domínio específico. Sua viabilidade prática deve-se em larga medida aos métodos de Parameter-Efficient Fine-Tuning (PEFT), em particular LoRA [1] e sua variante quantizada QLoRA [2], que reduzem drasticamente o custo computacional ao restringir o treinamento a um pequeno conjunto de adaptadores de baixo rank. Contudo, a especialização focada pode comprometer o desempenho do modelo em tarefas externas ao domínio de treino, fenômeno conhecido como esquecimento catastrófico ou regressão de capacidade.

A magnitude e a direção desse *trade-off*, entretanto, dependem fortemente de fatores arquiteturais e de hiperparâmetros que ainda são objeto de investigação empírica. Neste trabalho, conduzimos uma avaliação quantitativa desse *trade-off*. Treinamos o modelo Qwen2.5-3B-Instruct [5] via QLoRA para Text-to-SQL no benchmark Spider [3], medindo o ganho de desempenho na tarefa-alvo por meio de uma métrica de *Execution Accuracy* customizada, e simultaneamente avaliando a

variação de capacidade geral em uma suíte de 150 questões do MMLU [4] cobrindo STEM, Humanidades e Ciências Sociais. Comparamos duas configurações de hiperparâmetros, mantendo demais parâmetros idênticos para isolar o efeito da taxa de aprendizado.

Os resultados revelam ganho substancial de 14,89 pontos percentuais na Execution Accuracy e, em contraposição à hipótese inicial, pequeno *aumento* (não regressão) na acurácia agregada do MMLU. Analisamos as razões desse comportamento e discutimos suas implicações para o desenvolvimento de LLMs comerciais especializados.

O restante do trabalho está organizado como segue. A Seção II detalha o pipeline experimental, a configuração do LoRA e a métrica customizada; a Seção III apresenta os resultados quantitativos; a Seção IV discute os achados; a Seção V aborda reprodutibilidade e contaminação de dados.

II. METODOLOGIA

A. Modelo Base e Datasets

Adotamos o modelo Qwen2.5-3B-Instruct [5] como modelo base, com aproximadamente 3,09 bilhões de parâmetros. Trata-se de um modelo *instruct/chat* bem alinhado a tarefas estruturadas em forma de prompt *few-shot*. Para a tarefa-alvo, utilizamos o *training split* do Spider (concatenando `train_spider.json` e `train_others.json`, totalizando 8.659 exemplos) para o *fine-tuning*, e o *dev split* (1.034 exemplos) para a avaliação. O *dev split* não foi visto pelo modelo em nenhum momento do treinamento.

Para a avaliação de generalização, construímos uma suíte fixa de 150 questões do MMLU, com 50 questões por categoria, amostradas com semente fixa do *test split* de uma subcategoria representativa de cada eixo: `college_computer_science` (STEM), `philosophy` (Humanidades) e `high_school_macro_economics` (Ciências Sociais).

B. Pipeline de Dados

Adotamos um pipeline determinístico em três etapas. Primeiro, serializamos o esquema de cada banco como o conjunto de instruções `CREATE TABLE` extraídas do próprio SQLite via `sqlite_master`. Essa representação preserva chaves primárias, chaves estrangeiras e tipos, onde estes são elementos essenciais ao *schema linking*, isto é, à capacidade do modelo de mapear corretamente entidades da pergunta para tabelas e colunas do esquema.

Segundo, selecionamos três exemplos *few-shot* do *training split* de forma determinística: priorizamos bancos com menor número de tabelas (em ordem alfabética estável), de modo a manter o prompt compacto. Os três exemplos são fixados em `few_shot.json` e utilizados de forma idêntica em todas as avaliações (Fases 2 e 4), garantindo comparabilidade.

Terceiro, formatamos os dados de treino e de avaliação no mesmo *chat template*, ou seja, havendo associação entre `system`, `user`, e `assistant`. O turno `user` contém a serialização do esquema e a pergunta; o turno `assistant` (alvo do treinamento) contém apenas a consulta SQL. O alinhamento de formato entre treino e avaliação evita descasamento de distribuição entre as duas etapas.

C. Configuração do LoRA

A Tabela I detalha a configuração de fine-tuning. Empregamos QLoRA com base em 4-bit (NF4 com *double quantization*, computação em FP16), adicionando adaptadores LoRA apenas nas projeções de atenção (`q_proj`, `k_proj`, `v_proj`, `o_proj`). O número total de parâmetros treináveis é 7.372.800, ou 0,24% do total do modelo. A loss é calculada exclusivamente sobre os tokens do turno `assistant` via o mecanismo `assistant_only_loss` do framework TRL, preservando o sinal de aprendizado na consulta SQL alvo.

Para satisfazer o requisito de experimentação com hiperparâmetros, comparamos duas configurações que diferem apenas na taxa de aprendizado: Config A com $lr = 2 \times 10^{-4}$ e Config B com $lr = 1 \times 10^{-4}$. Todos os demais hiperparâmetros (LoRA, batch, scheduler, otimizador, sementes, dados, formato de prompt) são idênticos entre as duas, isolando o efeito da taxa de aprendizado.

D. Métrica de Execution Accuracy

Implementamos a métrica de Execution Accuracy como uma classe customizada que herda de `deepeval.metrics.BaseMetric`. A escolha do DeepEval [6] como framework de avaliação padroniza a interface para integração com testes automatizados *pytest* na Fase 4.

A lógica interna do método `measure` executa um pipeline de cinco etapas. **(1) Extração robusta da SQL.** A saída bruta do modelo passa por uma cadeia de filtros: remoção de blocos markdown (`"`sql ...`"`), recorte a partir do primeiro `WITH` ou `SELECT`, encerramento no primeiro `;` ou em parágrafo em branco, e descarte de linhas finais que não contenham tokens SQL. Este procedimento é necessário, pois o modelo-base frequentemente intercala explicações

Tabela I
CONFIGURAÇÃO DO LoRA E HIPERPARÂMETROS DE TREINAMENTO.

Parâmetro	Valor
rank (r)	16
α	32
dropout	0,05
target_modules	<code>q_proj</code> , <code>k_proj</code> , <code>v_proj</code> , <code>o_proj</code>
quant_type	NF4 (double quant.)
compute dtype	FP16
batch / grad. accum.	1 / 8 (batch efetivo 8)
max_length	1024 tokens
otimizador	<code>paged_adamw_32bit</code>
scheduler	cosine, warmup 3%
épocas	1 (ambas configurações)
lr (Config A / B)	2×10^{-4} / 1×10^{-4}
seed	42
GPU	Tesla T4 (16 GB)
parâmetros treináveis	7.372.800 (0,24% do total)

em prosa. **(2) Conexão somente-leitura** ao banco SQLite via URI `file:...?mode=ro`, impedindo escrita acidental. **(3) Execução protegida** de ambas as consultas (gerada e gold) sob `try/except`, com timeout de 30 segundos: erros de sintaxe ou de execução contabilizam falha (score 0,0). **(4) Normalização** dos resultados com arredondamento de floats a 6 casas decimais para eliminar ruído numérico. **(5) Comparação** dos conjuntos de resultados como *multiconjuntos* (`collections.Counter`), o que torna a métrica insensível à ordem das linhas e respeita multiplicidades. Quando a consulta gold contém `ORDER BY` (detectado por expressão regular), a comparação passa a ser posicional, respeitando a ordem.

A métrica retorna 1,0 em caso de equivalência semântica e 0,0 em caso de divergência ou erro. A comparação é sensível à ordem de *colunas*; o avaliador oficial do Spider (*test-suite evaluation*) é mais permissivo nesse aspecto. Mantemos essa decisão consciente para simplificar a implementação. A mesma instância da métrica é utilizada para o baseline (Fase 2) e para os modelos fine-tuned (Fase 4), garantindo comparabilidade integral.

E. Avaliação MMLU

Para a avaliação de regressão (Fase 5), seguimos o protocolo padrão do MMLU em modo 5-shot. Para cada subcategoria, utilizamos as 5 questões do *dev split* como exemplares de contexto, sendo exemplares idênticos para o modelo-base e os modelos *fine-tuned*, isolando o efeito do treinamento. As 50 questões de teste são amostradas deterministicamente do *test split* com semente fixa. Construímos o prompt no formato canônico do MMLU (texto contínuo, sem chat template), e empregamos decodificação gulosa. A predição é a primeira letra A–D extraída por expressão regular nos primeiros tokens gerados. Ressaltamos que falhas em produzir uma letra válida contabilizam erro.

III. RESULTADOS

A. Baseline (Fase 2)

O modelo Qwen2.5-3B-Instruct, sem treino adicional, atinge 0,4913 de Execution Accuracy no *dev split* completo (508 acertos em 1.034 exemplos). O valor é compatível com a literatura para modelos da ordem de 3B parâmetros com prompt *few-shot* e indica que o modelo dispõe, já no pré-treino, de capacidade não-trivial para Text-to-SQL, mostrando consistência com o fato de que código e SQL são massivamente representados nos corpora de pré-treino atuais. Entre os 20 bancos do *dev split*, observamos forte heterogeneidade: bancos como *car_1*, *real_estate_properties* e *tvshow* apresentam acurácias abaixo de 25%, enquanto outros, como *singer*, ultrapassam 65%.

B. Fine-Tuning (Fase 4)

A Tabela II resume o desempenho das duas configurações fine-tuned em comparação com o baseline. Ambas superaram o baseline com margem substancial: a Config A obtém ganho absoluto de 14,89 pp (relativo de 30,3%), e a Config B obtém 12,57 pp (25,6%). A Config A supera a Config B em 2,32 pp, sugerindo que a taxa de aprendizado mais alta foi mais efetiva em uma única época. A Fig. 1 apresenta a comparação visual.

Tabela II
EXECUTION ACCURACY (EA) NO SPIDER *dev split* (1.034 EXEMPLOS).

Modelo	EA	Δ (pp)	Δ (% rel.)
Baseline (Qwen2.5-3B)	0,4913	–	–
Fine-tuned config A	0,6402	+14,89	+30,3
Fine-tuned config B	0,6170	+12,57	+25,6

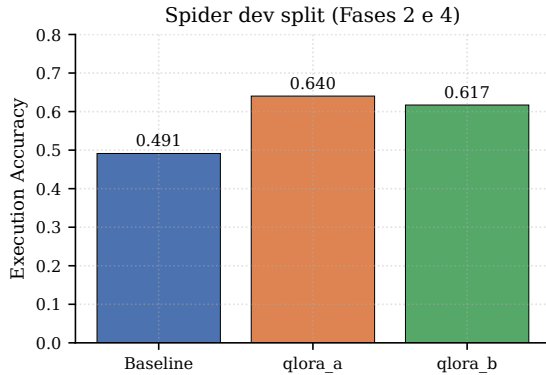


Figura 1. Execution Accuracy no Spider dev split.

C. Heterogeneidade entre bancos

O ganho médio mascara variabilidade significativa: na Config A, o ganho varia de +33,3 pp (*singer*, *voter_1*) até -7,1 pp (*pets_1*). A Tabela III mostra os cinco bancos com maior ganho e dois bancos onde o *fine-tuning* não ajudou ou prejudicou. Esse comportamento heterogêneo mostrou que o *fine-tuning* generalizado a partir do training split não se traduz uniformemente em ganho entre todos os esquemas do dev.

Tabela III
BANCOS COM MAIOR GANHO E REGRESSÃO (CONFIG A).

Banco	Base	Conf. A	Δ (pp)
singer	0,667	1,000	+33,3
voter_1	0,600	0,933	+33,3
network_1	0,500	0,821	+32,1
course_teach	0,500	0,800	+30,0
museum_visit	0,389	0,667	+27,8
dog_kennels	0,512	0,512	0,0
pets_1	0,619	0,548	-7,1

D. Regressão de Capacidade (Fase 5)

A Tabela IV apresenta a acurácia no MMLU 5-shot, por categoria e agregada. Contrariando a hipótese inicial de regressão, ambos os modelos fine-tuned mantêm ou superam o baseline em todas as categorias. A acurácia agregada da Config A sobe de 0,640 para 0,680 (+4,0 pp), e o ganho é mais pronunciado em STEM (+8,0 pp). A Config B apresenta variação positiva mas menor (+1,3 pp agregado), sem regressão em nenhuma categoria. A Fig. 2 ilustra os resultados.

Tabela IV
ACURÁCIA NO MMLU 5-SHOT (50 QUESTÕES POR CATEGORIA).

Categoria	Base	Conf. A	Conf. B	Δ A	Δ B
STEM	0,620	0,700	0,640	+8,0	+2,0
Humanidades	0,560	0,580	0,580	+2,0	+2,0
Ciências Sociais	0,740	0,760	0,740	+2,0	0,0
Agregado	0,640	0,680	0,653	+4,0	+1,3

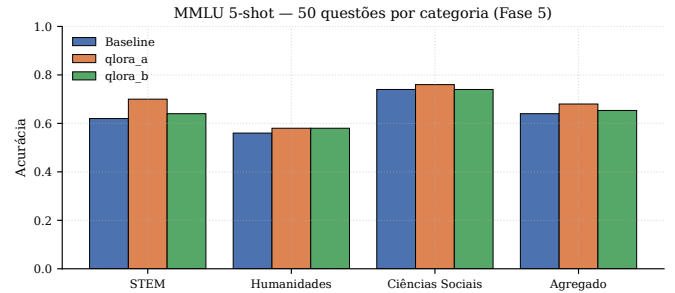


Figura 2. Acurácia no MMLU por categoria. Nenhuma categoria apresentou regressão; STEM concentrou o maior ganho.

E. Análise de Erros

Selecionamos três exemplos de casos de regressão: situações em que o baseline acertou mas a Config A errou. Tais casos são mais informativos do que falhas pré-existentes do modelo-base, pois revelam erros *introduzidos pelo treinamento*.

a) Exemplo 1 – Sobre-engenharia (banco *pets_1*):

Para a pergunta “Find the weight of the youngest dog”, a consulta gold é direta sobre a tabela *pets*:

```
SELECT weight FROM pets ORDER BY pet_age LIMIT 1
```

A Config A gerou uma consulta envolvendo *join* com *has_pet* e subconsulta sobre *student*, filtrando *pettype='dog'*, caracterizando ser uma interpretação semanticamente plausível, mas que retorna conjunto vazio no banco real. Notavelmente, *pets_1* é um dos bancos utilizados como exemplo *few-shot* durante o treino, sugerindo que o modelo aprendeu quando envolver *student* no esquema, aplicando o gatilho na situação errada.

b) *Exemplo 2 – Aliasing inconsistente (banco car_1):*

Para a pergunta sobre número de países por continente, o modelo gerou consulta quase idêntica à gold, mas substituiu *T1.ContId* por *T2.ContId* no *SELECT*. Como *T2* é a tabela *countries* e não possui a coluna *ContId*, o SQLite estoura erro de coluna inexistente. O erro evidencia uma limitação de raciocínio simbólico, isto é, o modelo conhece o esquema, mas perde controle do *aliasing* em consultas multi-tabela.

c) *Exemplo 3 – Literal sensível a case (banco flight_2):* Perguntado sobre a abreviação de “Jetblue Airways”, o modelo produziu *WHERE Airline = "Jetblue Airways"*. A tabela armazena “JetBlue Airways” (com J e B maiúsculos), e a comparação no SQLite é sensível ao caso, retornando conjunto vazio. O erro não é estrutural, mostrando ser apenas a transcrição literal da grafia da pergunta, que diverge da forma canônica armazenada na base.

Em conjunto, os três exemplos sugerem que o *fine-tuning* deslocou o perfil de erros do modelo: do baseline, onde predominavam falhas sintáticas grosseiras (consultas malformadas, referências a tabelas inexistentes), para o fine-tuned, onde predominam erros semânticos mais sutis (sobre-interpretação, *aliasing*, literais sensíveis a case). Essa transição é uma melhora qualitativa, pois erros sintáticos são triviais para um sistema detectar, enquanto erros semânticos exigem verificação no banco. No entanto, isto mostra que modelos de 3B parâmetros com QLoRA leve não atingem competência plena em Text-to-SQL.

IV. DISCUSSÃO

A. O ganho justifica a perda?

A pergunta central do enunciado, formulada sob a hipótese de que o fine-tuning causaria regressão em tarefas externas, encontra resposta empírica direta nos resultados, visto que não houve perda. Ambos os modelos *fine-tuned* apresentam ganho em ambos os eixos, tanto em especialização na tarefa-alvo (Spider), quanto na generalização na tarefa de controle (MMLU). A Fig. 3 ilustra essa configuração, pois os dois modelos *fine-tuned* ocupam o quadrante superior-direito do plano especialização-generalização, indicando dominância de Pareto sobre o baseline.

Esse achado contraria a expectativa estabelecida pela hipótese inicial, mas é consistente com observações recentes na literatura sobre PEFT. A magnitude do ganho de especialização (+14,89 pp em Execution Accuracy) é substancial e tem valor prático imediato, sem nenhum custo aparente em capacidade geral nas categorias avaliadas.

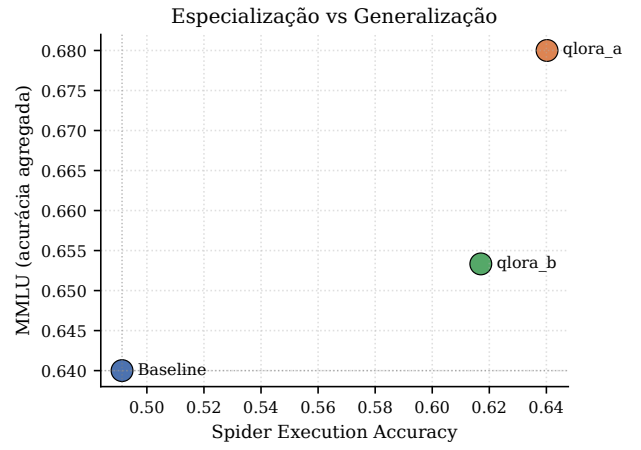


Figura 3. Trade-off especialização (Spider) vs. generalização (MMLU). As linhas tracejadas marcam o baseline; ambos os modelos fine-tuned ficam no quadrante superior-direito, indicando ganho em ambos os eixos.

B. Fatores que explicam a ausência de regressão

Identificamos quatro fatores que, em conjunto, plausibilizam a ausência de esquecimento catastrófico neste cenário.

Uso de baixa fração de parâmetros treináveis: Apenas 0,24% dos parâmetros do modelo foram ajustados. Restringir as atualizações às projeções de atenção, com rank baixo ($r = 16$), limita drasticamente o quanto o conhecimento geral do modelo pode ser sobrescrito. As representações pré-treinadas, distribuídas ao longo de 3,09 bilhões de pesos congelados, permanecem intactas.

Congelamento dos pesos do modelo-base em 4-bit: A quantização NF4 do modelo-base não é apenas uma economia de memória, mas ela também atua como uma forma de proteção contra sobrescrita. Mesmo se a magnitude dos gradientes propagasse efeitos ao modelo-base, a representação 4-bit reintroduz quantização a cada *forward pass*, descartando perturbações de pequena magnitude.

Treinamento com uma única época: Ambas as configurações treinaram por exatamente uma época sobre 8.659 exemplos. Treinos curtos limitam o sobreajuste (*overfitting*) na tarefa-alvo e, por extensão, a deformação do espaço de representação.

Visualizar geração de código SQL como tarefa estruturada: Diferentemente de tarefas que reforçam vieses estilísticos (escrita criativa, sumarização), o treinamento em SQL pode reforçar capacidade de raciocínio formal, caracterizando-se ser uma habilidade transferível. O ganho observado em STEM (+8 pp) é o mais expressivo entre as categorias do MMLU, e essa categoria é justamente a que mais beneficia de raciocínio simbólico estruturado, consistente com essa interpretação.

C. Comparação entre Configurações

A Config A ($lr = 2 \times 10^{-4}$) domina a Config B ($lr = 1 \times 10^{-4}$) em ambos os eixos: melhor Execution Accuracy (0,6402 vs 0,6170) e melhor acurácia MMLU agregada (0,680 vs 0,653). Em uma única época, a taxa de aprendizado mais alta foi mais efetiva, sem indícios de instabilidade ou

de *overfitting*, visto que a loss desce monotonicamente e a acurácia de *tokens* permanece estável ao longo do treino. Isso sugere que, para o contexto de QLoRA em modelos pequenos e treinamentos curtos, taxas relativamente altas são preferíveis a taxas conservadoras.

D. Heterogeneidade entre bancos

A variabilidade do ganho entre bancos (de +33 pp a -7 pp) revela que o *fine-tuning* não opera uniformemente sobre o espaço de esquemas. Os bancos com maior ganho (*singer*, *voter_1*, *network_1*) tendem a apresentar esquemas simples e padrões de consulta convencionais que aparecem frequentemente no *training split*. Os bancos onde o *fine-tuning* regrediu (notavelmente *pets_1*) sugerem situações em que o modelo aprende heurísticas específicas no treino que se aplicam erroneamente a outros casos. O fato de *pets_1* ser também um dos bancos usados como exemplo *few-shot* sugere possível interação entre escolhas de pré-processamento e o resultado do treinamento, constatando ser um ponto que justificaria investigação adicional.

E. Implicações práticas

Os achados têm implicações relevantes para o desenvolvimento de LLMs comerciais especializados. Primeiro, técnicas leves de PEFT em modelos abertos de pequeno porte (3B) são uma alternativa viável a APIs proprietárias para tarefas estruturadas como Text-to-SQL, com custo de treinamento da ordem de poucas horas em GPUs de consumidor. Em seguida, o fato de não observarmos regressão sugere que a preocupação com esquecimento catastrófico pode ser superestimada para configurações típicas de QLoRA. Entretanto, isso não isenta o desenvolvedor de auditar o modelo em tarefas relevantes ao seu caso de uso. Por fim, a heterogeneidade do ganho banco-a-banco sugere que avaliações agregadas (acurácia média) podem mascarar regressões em subdomínios específicos, o que recomenda análises desagregadas em produção.

V. REPRODUTIBILIDADE E CONTAMINAÇÃO DE DADOS

A. Reprodutibilidade

Toda a análise foi projetada para reprodutibilidade integral. Fixamos a semente em 42 (*seed* = 42) em todos os pontos estocásticos do pipeline: inicialização dos pesos LoRA, ordem de batches no treinamento, dropout, amostragem do MMLU, e demais operações pseudo-aleatórias do PyTorch e do NumPy. A semente é também propagada ao `SFTConfig` do framework TRL, garantindo determinismo no treinamento. Toda geração de texto para fins de avaliação utiliza decodificação gulosa (`do_sample=False`, `num_beams=1`), garantindo que saídas sejam reproduzíveis entre execuções. A seleção de exemplos *few-shot* é determinística por critério ordinal (bancos com menor número de tabelas, em ordem alfabética), eliminando dependência de *seed* na escolha. As versões exatas de todas as bibliotecas utilizadas estão fixadas em `requirements.txt`, obtido via `pip freeze` do ambiente Colab que produziu os resultados reportados.

B. Contaminação de Dados

Tanto o Spider (publicado em 2018) quanto o MMLU (publicado em 2021) integram corpora amplamente utilizados no treinamento de LLMs contemporâneos. É razoável presumir que parte ou totalidade desses *benchmarks* tenha sido vista pelo Qwen2.5 durante seu pré-treino. Essa contaminação tem três implicações para a interpretação dos nossos resultados.

Inicialmente, a Execution Accuracy do baseline (0,4913) pode estar parcialmente inflada por exposição prévia a exemplos do *dev split* ou a consultas semanticamente próximas. Segundo, o ganho de 14,89 pp observado no *fine-tuning* pode subestimar o ganho real, pois parte do conhecimento sobre Spider já estava no modelo, devido ao fato de que o *fine-tuning*, neste caso, refinaria capacidade pré-existente em vez de adquiri-la do zero. Por último, a ausência de regressão no MMLU deve ser interpretada com cautela, ou seja, se o modelo já viu as questões no pré-treino, sua capacidade de respondê-las pode ser robusta a perturbações leves dos pesos, mascarando fragilidade que apareceria em tarefas fora da distribuição.

REFERÊNCIAS

- [1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-Rank Adaptation of Large Language Models,” in *Proc. ICLR*, 2022.
- [2] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” in *Proc. NeurIPS*, 2023.
- [3] T. Yu *et al.*, “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task,” in *Proc. EMNLP*, 2018, pp. 3911–3921.
- [4] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Measuring Massive Multitask Language Understanding,” in *Proc. ICLR*, 2021.
- [5] Qwen Team, “Qwen2.5 Technical Report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [6] Confident AI, “DeepEval: The Open-Source LLM Evaluation Framework,” <https://github.com/confident-ai/deepeval>, 2024.